Guía de trabajo en grupo

Producto de la Unidad 1 — Matriz de decisión tecnológica · Bases de Datos III

Cada sección trae un recordatorio corto y un ejemplo, si algo no les quedó claro en la explicación en vivo, esto les debería alcanzar para seguir solos.

> **Recordatorio rápido: familias y motores**
>
> MongoDB — documental: documentos tipo JSON sin esquema fijo. Sirve cuando los datos varían mucho entre registros (catálogos).
>
> Redis — clave-valor: todo en memoria RAM, una clave que apunta a un valor. Sirve cuando la velocidad es crítica (cachés, sesiones).
>
> Neo4j — grafos: nodos y relaciones guardadas como conexiones directas. Sirve para redes y recomendaciones.
>
> Cassandra — columnar: datos distribuidos en muchos servidores. Sirve para volúmenes de escritura enormes (sensores, logs).
>
> PostgreSQL — relacional (SQL): tablas fijas con filas y columnas. Sirve cuando la consistencia es innegociable (pagos, transacciones).

Nombres del grupo: Joan Santiago Benavides Sánchez

Fecha:12/10/2026

---

1. Caso real

Plantilla: "[Sistema] que maneja [tipo de datos] para [quién lo usa]". Tiene que ser específico, no sirve "una empresa" o "una app".

> **Ejemplo**
>
> "Plataforma de streaming de música que maneja catálogo de canciones, historial de reproducción y recomendaciones, para usuarios finales."
>
> Si no tienen un caso propio, tomen uno del banco de 8 casos de respaldo.

Una plataforma para una alcaldía o curaduría. Reporte de daños en edificios/vias y seguimiento de cuadrillas de reparación

---

2. Tabla comparativa de motores candidatos

Elijan UNA parte de su sistema (no todo a la vez). Para esa parte, elijan 2-3 motores candidatos y califiquen cada criterio como Alta/Media/Baja, con una justificación de una línea.

> **¿Qué pregunta cada criterio?**

Consistencia — ¿qué tan grave es que el dato esté desactualizado por un instante? (grave, como en dinero → SQL; no grave, como un contador de likes → NoSQL puede.)

Latencia — ¿qué tan rápido necesito la respuesta: milisegundos, o puede tardar un poco más?

Escalabilidad — ¿cuánto va a crecer esto: cientos, miles, millones de registros?

Costo operativo — ¿cuánto cuesta mantener y operar ese motor a ese volumen?

Criterio	Motor 1: Mongodb	Motor 2: Redis	Motor 3: Neo4j
Consistencia	**Media.** Permite registrar reportes de daños y actualizar su estado.	**Alta.** Sus operaciones son atómicas y permite leer y actualizar datos rápidamente	**Alta.** Ofrece transacciones útiles para mantener relaciones y actualizaciones coherentes.
Latencia	**Media.** Es adecuado para consultar reportes, fotografías y datos de las cuadrillas	**Baja.** Es muy rápido para consultar información temporal.	**Baja.** Puede consultar rápidamente las relaciones entre daños, edificios, vías, cuadrillas y órdenes de trabajo.
Escalabilidad	**Alta.** Puede aumentar para manejar miles o millones de reportes mediante distribución de datos.	**Alta.** Puede distribuir la carga y atender muchas consultas, aunque depende de la memoria disponible y de la arquitectura.	**Media.** Es muy eficiente para analizar relaciones complejas.
Costo operativo	**Medio.** Puede funcionar con costos moderados si se utiliza una instalación sencilla o un servicio administrado.	**Medio.** Mantener los datos en memoria puede resultar costoso cuando aumenta el volumen	**Alto.** Puede necesitar personal especializado y recursos adicionales para administrar la base de datos.

Ejemplo de referencia (catálogo de canciones de una app de streaming) — la suya no tiene que verse igual, es solo para que vean cómo se llena:

Criterio	MongoDB (documental)	PostgreSQL (SQL)	Cassandra (columnar)
Consistencia	Baja — eventual, aceptable en un catálogo	Alta — más de lo que este caso exige	Media — eventual, ajustable

Criterio	MongoDB (documental)	PostgreSQL (SQL)	Cassandra (columnar)
Latencia	Media — buena para lecturas frecuentes	Media-alta — JOINs con metadatos variables cuestan	Baja — pensada para escritura masiva
Costo operativo	Medio	Medio — paga funciones que aquí no se usan	Alto — más nodos para tolerancia a fallos

3. Justificación final

Plantilla: "Para _____, elegí _____ porque _____ es la prioridad, evidenciado por _____." Tiene que citar el criterio Y una evidencia concreta de su caso — no basta con decir "uso MongoDB".

> **Ejemplo**
>
> "Para el catálogo de canciones, elegí MongoDB porque la variedad de metadatos es alta y cambia por género — un esquema fijo obligaría a columnas vacías o tablas separadas por tipo de contenido, evidenciado en que un track de jazz y uno electrónico casi no comparten atributos."

Motor 1 MongoDB Para una plataforma de reportes de daños en edificios y vías, elegí MongoDB porque la flexibilidad de los datos es la prioridad, evidenciado en que cada reporte puede tener información diferente, como fotografías, ubicación, descripción del daño, nivel de gravedad y estado de reparación.

Motor 2 Redis Para una plataforma de seguimiento de cuadrillas de reparación, elegí Redis porque la baja latencia es la prioridad, evidenciado en que la alcaldía necesita consultar rápidamente el estado de las reparaciones y la ubicación más reciente de las cuadrillas para coordinar su trabajo.

Motor 3 Neo4j Para una plataforma de gestión de daños y reparaciones de una alcaldía, elegí Neo4j porque las relaciones entre los datos son la prioridad, evidenciado en que un reporte puede estar relacionado con una vía o edificio, una cuadrilla asignada y una orden de reparación, lo que permite identificar conexiones y organizar mejor las intervenciones.

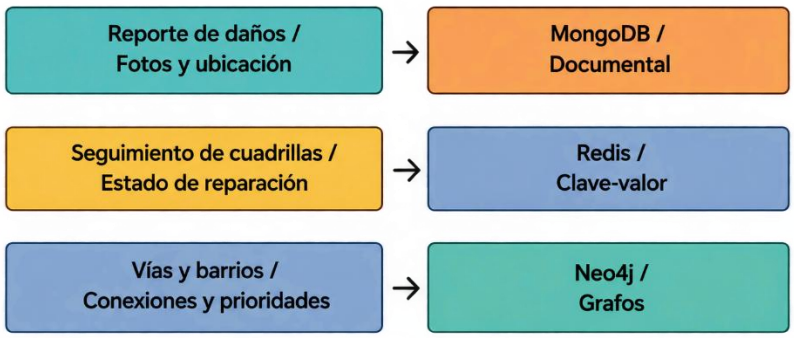
---

4. Arquitectura inicial

Pregunta guía: si tu aplicación tuviera 3 pantallas principales, ¿qué pantalla llama a qué base de datos? No necesitan UML formal, un diagrama de bloques simple basta.

> **Ejemplo**

Pantalla de búsqueda → MongoDB (catálogo). Pantalla de reproductor → Redis (qué está sonando). Pantalla de recomendaciones → Neo4j (grafo de gustos). Tres cajas, tres flechas.



5. Repositorio Git

¿Por qué ahora, aunque no haya código? Ese primer commit ya cuenta como evidencia de trabajo desde el día uno — en la Unidad 8 van a necesitar el historial completo.

☐ Repositorio creado en GitHub

☐ git clone — traje el repositorio a mi computador

☐ git add . — marqué los archivos para guardar

☐ git commit -m "..." — guardé los cambios con un mensaje

☐ git push — subí los cambios a GitHub

URL del repositorio: https://github.com/jsbenavides21-byte/Base-De-Datos-Plataforma-Infraestructura-Da-ada.git